



Projeto Final de TR2 - 2026_1

PROJETO FINAL

Streaming Adaptativo com Controle de Banda

Adaptive Bitrate (ABR) sobre HTTP

Disciplina: Teleinformática e Redes 2 (TR2)

Grupos de 3 alunos

1. Contextualização

O *streaming* de vídeo adaptativo é hoje a forma dominante de distribuição de conteúdo multimídia na Internet. Plataformas como YouTube, Netflix e Prime Video, dentre outras, utilizam variantes do protocolo DASH (*Dynamic Adaptive Streaming over HTTP*) para ajustar dinamicamente a qualidade do vídeo de acordo com as condições da rede.

Neste projeto, o grupo irá implementar um sistema de *streaming* adaptativo em Python, com um servidor HTTP com controle programático de banda e variação de atraso (*jitter*), e um cliente com algoritmos de seleção adaptativa de qualidade (ABR). O sistema suporta dois servidores com *failover* automático e gestão de *buffer* com estimativa de *continuous play*.

Por que este projeto?

Serviços como Netflix não transmitem vídeo continuamente: enviam segmentos de 2–4s via HTTP sobre TCP, e o cliente decide a qualidade de cada segmento com base no que observa da rede. Entender esse mecanismo exige dominar HTTP, TCP, medição de vazão, gestão de buffer e políticas de decisão adaptativa — todas as camadas do modelo vistas na disciplina.

2. Objetivos de Aprendizagem

Ao concluir este projeto, o grupo deverá ser capaz de:

- Descrever o funcionamento do HTTP em modo *chunked* e sua relação com o TCP
- Implementar um servidor HTTP com controle de taxa de transferência programático
- Medir vazão e variação de atraso (*jitter*) de uma conexão TCP a partir da aplicação
- Implementar gestão de buffer com estimativa de *continuous play* e detecção de *rebuffering*
- Projetar e implementar pelo menos três algoritmos ABR distintos
- Implementar *failover* automático entre servidores com base em *health check*
- Correlacionar eventos da aplicação (*rebuffering*, *failover*, troca de qualidade) com comportamentos TCP visíveis no Wireshark

3. Descrição do Sistema

3.1 Arquitetura

O sistema é composto por dois servidores e um cliente, todos implementados em Python puro (sem dependências externas obrigatórias):

Componente	Responsabilidade	Quem implementa
Servidor A (principal)	Expor <i>manifest</i> com lista de servidores; servir segmentos com <i>rate limiting</i> e <i>variação de atraso (jitter)</i> programáticos; expor <i>endpoint</i> de controle ao vivo	Fornecido como infraestrutura (vide item 8.2)
Servidor B (<i>fallback</i>)	Idêntico ao A, rodando em porta diferente; assume quando A falha	Fornecido como infraestrutura (vide item 8.2)
Cliente	Baixar <i>manifest</i> ; medir <i>vazão</i> e <i>variação de atraso (jitter)</i> ; gerir <i>buffer</i> ; aplicar ABR; detectar e executar <i>failover</i> ; coletar métricas em CSV	Implementado pelo grupo

3.2 Manifest v2.0

O *manifest* é um JSON retornado pelo servidor ao cliente na inicialização. Ele lista todos os servidores disponíveis (com prioridade) e todas as representações de qualidade. O cliente deve ler o *manifest* e usá-lo para todas as decisões de ABR e *failover*.

Qualidade	Bitrate (kbps)	Tamanho do segmento	Cenário típico
240p	200	~25 KB	Conexão muito limitada (< 300 kbps)
360p	400	~50 KB	Conexão ruim (300–600 kbps)
480p	600	~75 KB	Conexão média (600–900 kbps)
720p	1000	~125 KB	Conexão boa (900–1200 kbps)
1080p	1200	~150 KB	Conexão ótima (> 1200 kbps)

4. Tarefas Obrigatórias

Tarefa 1 — Política 1: Cliente funcional com Baseline (ABR Rate-Based)

Implementação do cliente básico com o algoritmo ABR mais simples: *Rate-Based* puro. Este será o baseline contra o qual as melhorias serão comparadas.

- Baixar e fazer o *parser* do *manifest* JSON do servidor;
- Medir *vazão* por segmento: bytes recebidos / tempo decorrido;
- Implementar *Rate-Based* ABR: seleciona a maior qualidade cujo *bitrate* seja inferior a *vazão* média recente, com fator de segurança;
- Implementar *BufferManager*: estimar o nível do buffer em segundos de conteúdo disponível, calculando $buffer += segment_duration$ a cada segmento recebido e $buffer -= tempo_real_decorrido$ continuamente

- Registrar no CSV se o buffer era suficiente para *continuous play* no momento de cada decisão ABR (campo *buffer_can_play*)
- Registrar métricas completas em CSV por segmento: *vazão*, *variação de atraso (jitter)*, nível do buffer, qualidade, *rebuffering*
- Gerar gráficos de vazão e qualidade ao longo dos segmentos

Importante sobre o baseline

O Rate-Based puro tem deficiências conhecidas: oscila em redes instáveis, não antecipa tendências e pode causar *rebuffering*. **Parte central do projeto é o grupo identificar essas deficiências empiricamente e propor soluções nas tarefas seguintes.**

Tarefa 2 — Política 2: Análise de deficiências + Política 2 + Failover

Com base nos dados coletados na Tarefa 1, o grupo deve identificar deficiências concretas do baseline e implementar uma segunda política ABR que enderece pelo menos uma delas. Nesta tarefa também é implementado o suporte a dois servidores com *failover* automático.

A Política 2 deve:

- Identificar deficiências do baseline com evidências numéricas dos CSVs (ex: oscilação de qualidade, *rebuffering*, tempo de adaptação)
- Implementar uma segunda política que resolva ao menos uma deficiência identificada
- Comparar as duas políticas sob os mesmos cenários de banda com gráficos sobrepostos

Failover e gestão de buffer:

- Ler a lista de servidores do *manifest* e implementar *failover* automático por prioridade
- Ao detectar falha no servidor atual: verificar disponibilidade do alternativo via *health check* (GET */health*), migrar e registrar o evento no CSV
- O campo *buffer_can_play* deve indicar se o buffer era suficiente para absorver a troca sem *rebuffering*
- Analisar no relatório: quanto tempo levou o *failover*, o buffer foi suficiente, qual qualidade foi selecionada imediatamente após a troca e por quê

Sugestões para a Política 2 (não obrigatórias)

Buffer-Based ABR: decide qualidade com base no nível do buffer, não na vazão direta. Rate-Based com histerese: só muda qualidade após N segmentos consecutivos confirmando a nova banda. Conservador com slow-start: começa sempre na menor qualidade e sobe gradualmente.

Tarefa 3 — Política 3 com componente estatístico ou heurístico

O grupo deve implementar uma terceira política que incorpore algum componente analítico para melhorar a predição da próxima qualidade. A política deve ser justificada com embasamento teórico mínimo e comparada com as duas anteriores.

- Obrigatório: ao menos um componente estatístico ou heurístico explicado e justificado
- Deve tratar explicitamente o cenário de variação de atraso (*jitter*) alto: como a variação no tempo de entrega dos segmentos afeta a decisão ABR e o nível do buffer
- O grupo deve argumentar com dados por que essa abordagem é superior ao baseline

Exemplos de abordagens aceitas (não obrigatórias):

- Média móvel exponencial (EWMA): vazão recente tem mais peso que o passado distante
- Detecção de tendência: se vazão está caindo consistentemente, antecipar redução de qualidade
- Modelo com desvio padrão: usa $\text{vazão_médio} - k \times \text{desvio_padrão}$ para estimativa conservadora
- Política híbrida: combina nível de buffer e vazão com pesos ajustáveis
- Penalidade de variação de atraso (*jitter*): reduz estimativa de vazão proporcionalmente ao variação de atraso (*jitter*) medido

Sobre o uso de ML

O uso de Machine Learning (scikit-learn, regressão, etc.) é permitido, mas não obrigatório. O que será avaliado é a qualidade da justificativa e a análise dos resultados — não a complexidade da técnica.

Tarefa 4 — Análise de camadas com Wireshark

Esta tarefa conecta a implementação com os conceitos teóricos das camadas de transporte e rede.

- Capturar o tráfego TCP da sessão com Wireshark (filtro: tcp.port == 8080)
- Correlacionar timestamps dos eventos TCP com decisões ABR registradas no CSV
- Capturar o momento do failover: identificar o TCP RST ou FIN do servidor A e o novo TCP SYN para o servidor B, correlacionando com o timestamp do evento no CSV
- Incluir no relatório prints anotados do Wireshark explicando a correlação

5. Cronograma e Entregas

Entrega	Semana	O que apresentar	Duração
Entrega 1	Semana 4	Cliente funcional com baseline rodando. Demo ao vivo: baixar 10 segmentos no servidor do professor, mostrar CSV gerado e gráfico de vazão.	10 min
Entrega 2	Semana 7	Política 2 + failover implementados. Demo: rodar baseline vs política 2 com mudança de banda ao vivo; derrubar o servidor A e mostrar failover automático para o B.	10 min
Apresentação Final	Semana 10	Todas as 3 políticas + failover + análise Wireshark. Relatório entregue no AVA antes da apresentação. Demo com cenário surpresa.	20–25 min

Cenário surpresa na apresentação final

O professor irá: (a) alterar a banda ao vivo, (b) introduzir variação de atraso (jitter), e (c) derrubar o servidor A. O grupo deverá explicar em tempo real: quanto tempo levou o failover, se o buffer foi suficiente para evitar rebuffering, qual política reagiu melhor à troca de servidor e onde no código essa decisão é tomada.

6. Relatório Final

O relatório deve ser entregue em PDF via Aprender até 48 horas antes da apresentação. Estrutura obrigatória:

1. Introdução e contextualização do problema
2. Descrição da arquitetura implementada (cliente, buffer manager, failover)
3. Algoritmo Baseline: descrição, pseudocódigo e gráficos

4. Análise de deficiências do baseline com evidências dos CSVs
5. Política 2: motivação, descrição e comparação com o baseline
6. Política 3: componente estatístico/heurístico, justificativa e resultados
7. Análise do *failover*: *timestamp* do evento, nível do buffer no momento da troca, impacto na qualidade
8. Análise Wireshark: prints anotados com correlação entre eventos TCP e decisões ABR
9. Discussão: qual política teve melhor desempenho e por quê (com dados)
10. Conclusão

O relatório deve conter obrigatoriamente gráficos gerados a partir dos CSVs da sessão de apresentação final. Os arquivos CSV devem ser entregues como anexo.

Gráficos obrigatórios (gerados pelo próprio grupo):

- Vazão medido + qualidade selecionada ao longo dos segmentos
- Nível do buffer ao longo do tempo, com eventos de *rebuffering* marcados
- Variação de atraso (*jitter*) EWMA ao longo do tempo
- Comparação das 3 políticas ABR lado a lado (mesma escala, mesmo cenário)
- Para quem implementou failover: linha vertical marcando o segmento do evento em todos os gráficos

7. Critérios de Avaliação

Critério	Peso	O que será avaliado
Entrega 1 — Baseline + buffer	15%	Cliente conecta, baixa segmentos, buffer manager funcional, CSV gerado com <code>buffer_can_play</code> , gráfico de vazão.
Entrega 2 — Política 2 + failover	20%	Deficiências identificadas com dados. Política 2 comparada. Failover automático demonstrado ao vivo.
Política 3 (estatística/heurística)	20%	Componente analítico justificado. Trata variação de atraso (<i>jitter</i>). Resultados comparados com as políticas anteriores.
Análise Wireshark	15%	Eventos TCP identificados corretamente. Failover correlacionado. Correlação com CSV documentada.
Relatório	15%	Clareza, estrutura, gráficos com dados da sessão real, discussão fundamentada com números.
Apresentação final + cenário surpresa	15%	Demo ao vivo, reatividade ao cenário surpresa (banda + variação de atraso (<i>jitter</i>) + failover), capacidade de explicar decisões do código.

8. Infraestrutura e Ambiente

8.1 Requisitos mínimos

- Python 3.6 ou superior (sem dependências externas obrigatórias)
- Wireshark instalado para a Tarefa 4
- `matplotlib` para gráficos (`pip install matplotlib`) — opcional, o grupo pode usar outra biblioteca

8.2 Servidor

O servidor estará disponível nos seguintes endereços, acessíveis pela Internet:

- **Servidor A (principal):** <http://137.131.178.229:8080>
- **Servidor B (fallback):** <http://137.131.178.229:8081>

O manifest com a lista de servidores e representações disponíveis pode ser consultado em

<http://137.131.178.229:8080/manifest>.

O código do servidor **não será disponibilizado**. O servidor é tratado como infraestrutura — os alunos implementam apenas o cliente. Durante as entregas e a apresentação final, o servidor estará ativo e acessível. Em caso de indisponibilidade, contate o professor.

8.3 Métricas registradas no CSV

O cliente deve registrar as seguintes métricas em CSV após cada segmento:

Campo	Descrição
segment	Número sequencial do segmento
timestamp	Horário do download (ISO 8601)
server_id	ID do servidor que atendeu este segmento (A ou B)
quality	Qualidade selecionada pelo ABR
bitrate_kbps	Bitrate nominal da representação
vazão_kbps	Vazão medido neste segmento
download_time_s	Tempo de download
variação de atraso (jitter)_network_ms	Variação de latência entre chunks do mesmo segmento
variação de atraso (jitter)_ewma_ms	EWMA da variação entre segmentos consecutivos
buffer_level_s	Nível estimado do buffer em segundos
buffer_can_play	1 se o buffer era suficiente para continuous play, 0 caso contrário
rebuffer_event	1 se ocorreu rebuffering neste segmento
stall_duration_s	Duração do stall em segundos (0 se não houve)
failover_total	Número acumulado de failovers até este segmento

9. Dicas Importantes

Dica 1 — Comece pelo mais simples

Na Entrega 1, faça o baseline e o buffer manager funcionarem corretamente antes de qualquer outra coisa. Um cliente que conecta, baixa, estima buffer e gera CSV vale mais que uma política complexa com bugs.

Dica 2 — Os dados são o projeto

O CSV é a base de toda a análise. Cada decisão ABR deve aparecer no CSV e ser explicada. No cenário surpresa, você deve apontar exatamente em qual segmento o cliente detectou a mudança de banda, e por que o buffer foi ou não suficiente durante o failover.

Dica 3 — Wireshark e o CSV falam a mesma língua

O timestamp no CSV e o timestamp no Wireshark são sincronizados (relógio do sistema). Use isso: quando o Wireshark mostra TCP RST do servidor A, qual segmento estava sendo baixado? O campo `server_id` mudou no próximo segmento? O `buffer_can_play` era 1 naquele momento?

Dica 4 — Compare com dados, não com opiniões

"Nossa política 3 é melhor" não é conclusão aceita. "Nossa política 3 reduziu os eventos de rebuffering de 4 para 1 e manteve `buffer_can_play=1` durante o failover, enquanto o baseline teve rebuffer de 1.0s" é conclusão aceita.

10. Integridade Acadêmica e Uso de IA

O uso de ferramentas de inteligência artificial é permitido, desde que:

- O grupo compreenda e saiba explicar todo o código submetido
- A análise das deficiências, a justificativa das políticas e as conclusões sejam baseadas nos dados reais coletados pelo grupo
- Os gráficos e CSVs do relatório final sejam da sessão de apresentação real

Atenção

Durante o cenário surpresa serão feitas perguntas diretas: onde no código a decisão ABR é tomada, por que o failover demorou X segmentos, o que teria acontecido se o buffer estivesse abaixo de 4s no momento da queda do servidor A. A incapacidade de responder implica desconto na nota de apresentação.

Bom projeto a todos!

Versão 01: 06/05/2026

Última atualização: quarta-feira, 6 mai. 2026, 07:55

Entre em contato



Siga nossas redes sociais



Você acessou como Leonardo Pereira Côrtes (Sair)

Resumo de retenção de dados

Baixar o aplicativo móvel.

Baixar o aplicativo móvel.